

Lecture 3.1: Python

XCS229 Machine Learning

[Lecturer (i.e. Tengu Ma or Chris Re)]

I. Python Basics Overview

This section provides an introduction to fundamental Python syntax within a Jupyter notebook environment. It covers how to execute code cells, assign values to variables, display output.

Running Code in a Jupyter Notebook:

- Windows/Linux: Press Shift + Enter to execute the current cell and move to the next one.
- Mac: Press Command + Enter to run the current cell without moving to the next one.

Variable Assignment

Variables can be assigned values easily, for example, `code = 229`. Calling the variable will return its value.

```
[1]: code = 229
```

```
[2]: code
```

229

Conditional Statements

Python uses `if`, `elif`, and `else` for control flow.

```
[3]: if code ==229:
      print('Hello CS229!')
      elif code == 230:
      print('That \'s deep learning!')
      elif code < 200:
      print('That is some undergraduate class')
```

```
else:
    print('Wrong class!')
```

Hello CS229!

Unlike, Python does not have a 'switch' statement.

II. Python Operators

This section discusses various Python operators, including logical, bitwise, arithmetic operators, loops, and functions explaining their usage with examples.

- **Logical Operators:** The logical operators in Python (and, or, not) are straightforward and can be used directly with variables without needing explicit comparisons.

```
[4]: true=True
     false=False
```

```
[5]: if true:
     print("It's true!")
     if not false :
     print("It's still true!")
     if true and not false:
     print("Anyhow, it's true!")
     if false or not true:
     print("True?")
     else:
     print("Okay, it's false now ....")
```

```
It's true!
It's still true!
Anyhow, it's true!
Okay, it's false now ....
```

- **Bitwise Operators:** & (and), | (or), and ~ (not), are all bitwise operators, which are particularly useful when working with NumPy arrays. Also, ^ means bitwise XOR in python.
- **Arithmetic Operators:** Python supports various arithmetic operations, including:

- Division (/): Produces a floating-point result.
- Modulus (%): Returns the remainder of a division.
- Exponentiation (^ or **): Used to calculate powers.
- Integer Division (//): Discards the decimal part and returns an integer.
- Addition (+), Subtraction (-), and Multiplication (*): Used for basic calculations.

Python automatically determines whether a variable is an integer or a float based on the operation performed.

```
[6]: print(5/2) # floating number division
      print(5%2) # remainder
      print(5**2) # exponentiation
      print(5//2) # integer division
      print(5 + 2) # addition
      print(5 - 2) # subtraction
      print(5 * 2) # multiplication
```

```
2.5
1
25
2
7
3
10
```

• **Loops:** Python provides two main types of loops:

- The **for** loop is commonly used with the **range()** function to iterate over a sequence of numbers.
For instance, **for i in range(5)** iterates from 0 to 4.

```
[7]: for i in range(5):
      print(i)
```

```
0
1
2
3
4
```

- The **while** loop continues execution as long as a specified condition remains true.

```
[8]: a=5
      while a>0:
          print(a)
          a-=1
```

```
5
4
3
2
1
```

We typically use *range* and *enumerate* for iterations. Python doesn't have commands like "a++" or "a-" as in C++. Instead, we write "a = a - 1" or "a -= 1" to decrement a value.

- **Functions:** Functions are defined using the **def** keyword, followed by the function name and parameters. After that, a colon is used, and the block of code inside the function is indented.

Python emphasizes indentation to denote code blocks, which can lead to errors if not managed correctly.

Additionally, python functions can default arguments, they to be at the end. Be very careful because forgetting that you have prevent you from debugging effectively.

```
[9]: def power(v, p=2):
      return v** p # How to return multiple values?
      print(power(10))
      print(power(10,3)) # specify explicit powers (power 3)
```

```
100
1000
```

Functions can support extra arguments. These can be passed on to another function or used directly:

- **args:** variable number of non-keyworded arguments
- **kwargs:** variable number of keyworded arguments

Imagine you want to sum several variables, say A, B, and C, but later you decide to add a new variable D. Instead of creating a new function for each possible combination, you can use ***args** and ****kwargs** to handle a variable number of arguments without having to redefine the function every time.

Steps in the example below:

- func2(*args, **kwargs): This function prints out the non-keyword arguments (args) and keyword arguments (kwargs).
- In func1, the *args allows us to pass any number of non-keyword arguments, while **kwargs allows us to pass keyword arguments.
- if 'power' in kwargs: Checks if the power keyword exists in the arguments and applies it if present; otherwise, it just returns the value v.

```
[10]: def func2(*args, **kwargs):  
        print(args)  
        print(kwargs)
```

```
[11]: def func1(v, *args, **kwargs):  
        func2(*args, **kwargs)  
        if 'power' in kwargs:  
            return v**kwargs['power']  
        else:  
            return v  
  
        print(func1(10, 'extra 1', 'extra 2', power=3))  
        print('-----')  
        print(func1(10,5))
```

```
('extra 1', 'extra 2')  
{'power': 3}  
1000  
-----  
(5,)  
{}  
10
```

III. Sample Python Data Types

The section discusses various data structures in Python, including strings, lists, tuples, sets, and dictionaries, highlighting their properties and practical applications. Python offers several built-in data types, each with unique characteristics and methods for efficient manipulation, making it easier to handle and organize data effectively.

- **Strings:** are essential for text manipulation, supporting operations like concatenation and formatting.

```
[12]: cs_class_code=229
```

```
[13]: # Concatenation
print('I like CS '+str(cs_class_code)+' a lot!')
print(f'I like CS {cs_class_code} a lot!')
```

```
I like CS 229 a lot!
```

```
I like CS 229 a lot!
```

```
[14]: # Formatting
print('I love CS229. (upper)'.upper())
print('I love CS229. (rjust 50)'.rjust(50))
print('We love CS229. (capitalize)'.capitalize())
print('      I love CS229. (strip)      '.strip())
```

```
I LOVE CS229. (UPPER)
```

```
I love CS229. (rjust 50)
```

```
We love cs229. (capitalize)
```

```
I love CS229. (strip)
```

f-string (f for formatting?) is new since Python 3.6 for easier variable interpolation. Embed values using {}.

```
[15]: print(f'{print} (print a function)')
print(f'{type(229)} (print a type)')
```

```
<built-in function print> (print a function)
```

```
<class 'int'> (print a type)
```

```
[16]: print('Old school formatting: {2}, {1}, {0:10.2F}'.
format(1.358, 'b', 'c')) # Fill in order of 2, 1, 0. For the
decimal number, fix at length of 10, round to 2 decimal
places
```

```
Old school formatting: c, b,      1.36
```

- This example demonstrates string formatting in Python to print values in a specific order.
- The order is determined using 2, 1, 0, where the numbers refer to indices inside `format()`.

- The first variable (0), which is 1.358, is formatted using 10.2F, meaning:
 - * The number is rounded to two decimal places (.2f).
 - * A total width of 10 characters is reserved for the number, ensuring proper alignment.
- When printing, the order is rearranged as follows:
 - * The third value ("c") is printed first.
 - * The second value ("b") comes next.
 - * The first value (1.358) is formatted and displayed as 1.36.
- **Lists:** are versatile data structures that can contain mixed data types. Operations such as appending, inserting, and list comprehension facilitate efficient data handling. List comprehensions allow for concise syntax to create lists based on existing lists or ranges.

```
[17]: list_1=['one', 'two', 'three']
      list_2=[1,2,3]
```

```
[18]: print(list_1)
      print(list_2)
```

```
['three', 'two', 'one']
[1, 2, 3]
```

```
[19]: list_2.append(4)
```

```
[20]: list_2
```

```
[1, 2, 3, 4]
```

```
[21]: list_2.insert(0, 'ZERO')
```

```
[22]: list_2 # if you back to the previous cell and run
      print(list_2) again, what will you see?
```

```
['ZERO', 1, 2, 3, 4]
```

List extension is just addition:

```
[23]: print(list_1 + list_2)
      list_1_temp=['a', 'b']
      list_1_temp.extend(list_2)
      print(list_1_temp)
```

```
['one', 'two', 'three', 'ZERO', 1, 2, 3]
['a', 'b', 'ZERO', 1, 2, 3]
```

But be very careful when you multiply a list!

```
[24]: print(list_1*3 + list_2)
      print([list_1]*3 + list_2)
```

```
['one', 'two', 'three', 'one', 'two', 'three', 'one',
'two', 'three', 'ZERO', 1, 2, 3]
[['one', 'two', 'three'], ['one', 'two', 'three'],
['one', 'two', 'three'], 'ZERO', 1, 2, 3]
```

`pprint`: module in Python provides a way to print complex nested structures (such as lists and dictionaries) in a well-organized and readable format. This is particularly useful when debugging or handling large datasets.

```
[25]: import pprint as pp
```

```
[26]: pp.pprint([list_1]*5 + list_2)
      pp.pprint([list_1]*2 + [list_2]*3)
```

```
[['one', 'two', 'three'],
['one', 'two', 'three'],
['one', 'two', 'three'],
['one', 'two', 'three'],
['one', 'two', 'three'],
'ZERO',
1,
2,
3]
[['one', 'two', 'three'],
['one', 'two', 'three'],
['ZERO', 1, 2, 3],
['ZERO', 1, 2, 3],
['ZERO', 1, 2, 3]]
```


Why use pprint?

- If a list contains many elements (e.g., 100+ items), standard `print()` output may be difficult to read.
- `pprint` helps in visualizing the nested structure and understanding the data at a glance.

List comprehension: allows you to create lists efficiently in one line using loops and conditions. This method makes your code cleaner, more readable, and save a lot of lines.

```
[27]: long_list=[i for i in range(9)]
      long_Long_list=[(i,j) for i in range(3) for j in range(5)]
      long_list_list=[[i for i in range(3)] for _ in range (5)]

      pp.pprint(long_list) # generates numbers from 0 to 8
      pp.pprint(long_Long_list) # creates tuples of i and j
      pp.pprint(long_list_list)# creates a 5x3 list with 0, 1, 2.
```

```
[0, 1, 2, 3, 4, 5, 6, 7, 8]
[(0, 0),
(0, 1),
(0, 2),
(0, 3),
(0, 4),
(1, 0),
(1, 1),
(1, 2),
(1, 3),
(1, 4),
(2, 0),
(2, 1),
(2, 2),
(2, 3),
(2, 4)]
[[0, 1, 2], [0, 1, 2], [0, 1, 2], [0, 1, 2], [0, 1, 2]]
```

List is iterable! You can loop over the elements in a list.

```
[28]: string_list=['a', 'b', 'c']
```

```
[29]: for s in string_list:
      print(s) # you can print each element of the list
```

a
b
c

Using `enumerate()`: With `for i, s in enumerate(string_list)`, you can also get the index of each element along with its value.

```
[30]: for i,s in enumerate(string_list):  
       print(f'{i}, {s}')
```

0, a
1, b
2, c

Slicing: with NumPy array (covered later), you can do this to multi-dimensional ones as well.

```
[31]: print(long_list[:5]) # First 5 elements  
       print(long_list[:-1]) # Excluding last element  
       print(long_list[4:-1]) # From 4th to second-to-last  
  
       long_list[3:5]=[-1,-2] # Replaces the 4th and 5th elements  
                               with -1 and -2  
       print(long_list)  
  
       long_list.pop() # Removes the last element from the list  
       print(long_list)
```

[0, 1, 2, 3, 4]
[0, 1, 2, 3, 4, 5, 6, 7]
[4, 5, 6, 7]
[0, 1, 2, -1, -2, 5, 6, 7, 8]
[0, 1, 2, -1, -2, 5, 6, 7]

Sorting list: but remember that sorting can be costly.

```
[32]: random_list=[3,12,5,6,8,2]  
       print(sorted(random_list))  
  
       random_list_2=[(3,'z'), (12,'r'), (5,'a'), (6,'e'), (8,'c'),  
                       (2,'g')]
```

```
print(sorted(random_list_2, key=lambda x: x[1])) #sorting
using 2nd variable in tuple
```

```
[2, 3, 5, 6, 8, 12]
[(5, 'a'), (8, 'c'), (6, 'e'), (2, 'g'), (12, 'r'), (3,
'z')]
```

Think first before copying be reference not be value.

```
[33]: orig_list=[[1,2], [3,4]]
      dup_list=orig_list # Reference assignment

      dup_list[0][1]='okay'
      pp.pprint(orig_list) # Changes reflected in orig_list
      pp.pprint(dup_list)
```

```
[[1, 'okay'], [3, 4]]
[[1, 'okay'], [3, 4]]
```

```
[34]: a= [[1,2,3]]*3 # Reference to same list 3 times
      b= [[1,2,3] for i in range(3)] # Creates separate lists
      a[0][1]=4
      b[0][1]=4
      print(a) # Affected all lists in a
      print(b) # Only affected first list in b
```

```
[[1, 4, 3], [1, 4, 3], [1, 4, 3]]
[[1, 4, 3], [1, 2, 3], [1, 2, 3]]
```

```
[35]: import copy
```

```
[36]: orig_list=[[1,2], [3,4]]
      dup_list=copy.deepcopy(orig_list) # Deep copy

      dup_list[0][1]='okay'
      pp.pprint(orig_list) # No changes to orig_list
      pp.pprint(dup_list)
```

```
[[1, 2], [3, 4]]
[[1, 'okay'], [3, 4]]
```

- **Tuples:** are similar to lists but are immutable, meaning their contents cannot be altered after creation. The content illustrates how to assign tuple elements to multiple variables simultaneously.

```
[37]: my_tuple=(10, 20, 30)
      my_tuple[0]=40
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[37], line 2
      1 my_tuple=(10, 20, 30)
----> 2 my_tuple[0]=40
```

```
TypeError: 'tuple' object does not support item
assignment
```

Split assignment makes your code shorter (also work for list).

```
[38]: a, b, c =my_tuple
      print(f"a={a}, b={b}, c={c}")
      for obj in enumerate(my_tuple):
          print(obj)
```

```
a=10, b=20, c=30
(0, 10)
(1, 20)
(2, 30)
```

- **Sets:** are described as unordered collections of unique elements, emphasizing their usefulness in eliminating duplicates.

```
[39]: my_set={i**2 for i in range(10)}
      print(my_set) #unordered, unique
```

```
{0, 1, 64, 4, 36, 9, 16, 49, 81, 25}
```

```
[40]: my_set_temp={i**2 % 5 for i in range(10)}
      print(my_set_temp)
```

```
{0, 1, 4}
```

- **Dictionaries:** are introduced as key-value pairs, facilitating data retrieval. The content explains how to iterate through dictionaries and update them.

```
[41]: my_dict={i: i**2 for i in range(10)}
      print(my_dict)
      print(my_dict.keys())
```

```
{0: 0, 1: 1, 2: 4, 3: 9, 4: 16, 5: 25, 6: 36, 7: 49, 8:
64, 9: 81}
dict_keys([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

```
[42]: my_dict={(5-i): i**2 for i in range(10)}
      print(my_dict)
      print(my_dict.keys())
```

```
{5: 0, 4: 1, 3: 4, 2: 9, 1: 16, 0: 25, -1: 36, -2: 49,
-3: 64, -4: 81}
dict_keys([5, 4, 3, 2, 1, 0, -1, -2, -3, -4])
```

Updating and/or adding content to a dictionary.

```
[43]: second_dist={'a': 10, 'b': 11}
      my_dict.update(second_dist)

      pp.pprint(my_dict)
```

```
{0: 0,
1: 1,
2: 4,
3: 9,
4: 16,
5: 25,
6: 36,
7: 49,
8: 64,
9: 81,
'a': 10,
'b': 11}
```

```
[44]: my_dict['new']=10
      pp.pprint(my_dict)
```

```
{0: 0,
1: 1,
2: 4,
3: 9,
4: 16,
5: 25,
6: 36,
7: 49,
8: 64,
9: 81,
'a': 10,
'b': 11,
'new': 10}
```

Here is how to iterate through a dictionary. And remember that dictionary is NOT sorted by key value.

```
[45]: for k, it in my_dict.items(): # similar to for loop over
      enumerate (list)
      print(k, it)
```

```
0 0
1 1
2 4
3 9
4 16
5 25
6 36
7 49
8 64
9 81
a 10
b 11
new 10
```

```
[46]: # Sorting keys by string order
      for k, it in sorted(my_dict.items(), key= lambda x:
      str(x[0])):
      print(k, it)
```

```
0 0
1 1
2 4
3 9
4 16
5 25
6 36
7 49
8 64
9 81
a 10
b 11
new 10
```

IV. Summary

The lecture serves as a comprehensive introduction to Python programming, emphasizing the language's simplicity and versatility. Key takeaways include:

- **Understanding Syntax:** Mastery of Python's syntax and control structures is foundational for effective programming.
- **Utilizing Data Structures:** Familiarity with Python's data structures enhances data management and manipulation capabilities.
- **Function Definitions:** Knowing how to define and use functions, including handling various types of arguments, is essential for writing reusable code.
- **Preparation for Numpy:** A solid grasp of Python basics is necessary for leveraging Numpy effectively in data science and machine learning applications.